

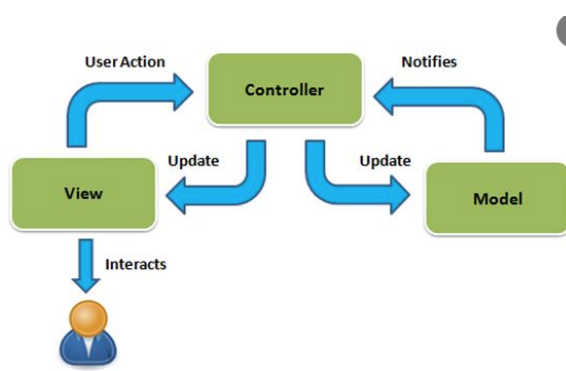
MVC

Separando funciones

Introducción

El patrón de arquitectura MVC, se refiere a una manera de organizar el código. La principal característica es separar el almacenamiento de los datos de la capa de visualización. Podéis ver la [lista de frameworks](#) que trabajan sobre este patrón

Utiliza tres componentes:



1. **Modelo**: encargado del manejo de los datos
2. **Vista**: encargada de la parte visual e interactiva con el usuario
3. **Controlador**: la lógica de la aplicación y enlace entre el Modelo y la Vista

La mejor manera de entender como funciona es con un ejemplo. Vamos a

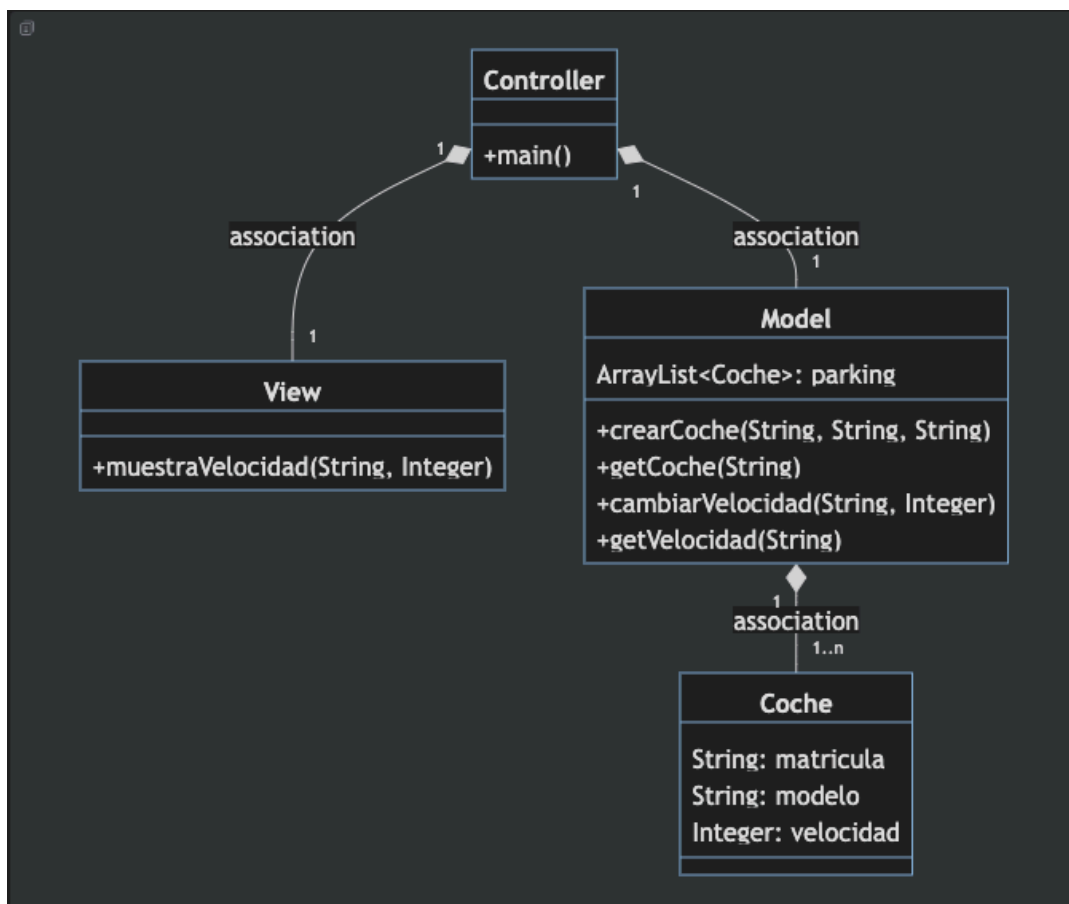
partir de un código que no está diseñado como MVC y lo iremos transformando hasta separa las capas.

Este patrón de arquitectura utiliza otro patrón de diseño, el [Observer](#), que es el necesario para notificar los cambios entre las tres partes. En principio no lo vamos a usar, dejando para más adelante este patrón.

Aplicación

Nuestra [aplicación](#) será muy sencilla, con apenas una clase de datos, y dos botones de interacción. Creará objetos coches y los dos botones serán para acelerar y frenar. La velocidad queremos verla en la interfaz gráfica.

Este es el diagrama de clase:



3 MVC

Controller

MVC es un patrón que se usa en prácticamente todos los lenguajes de programación, y se basa en objetos + eventos. Por lo tanto vamos a usar Java como el lenguaje para implementarlo.

Lo primero va a ser definir nuestra clase Controller (también la vista y el modelo serán clases).

Básicamente el Controller es nuestro código principal, el 'main', que lo hemos puesto en la clase principal.

Model

Vamos a por nuestros datos, es decir la `class Coche`

Queremos conservar la clase tal cual, nuestro modelo manejará esta clase (y otras que un futuro podremos añadir). Esta clase sería lo más parecido a los datos que tuviéramos en nuestra base de datos.

Los coches instanciados los vamos a almacenar en un array (`parking`) y de momento dos métodos uno para crear coches y otro para recuperarlo según la matrícula.

Este Model lo usará el Controller.

En el Controller podemos añadir los coches que queramos.

4 MVC

Podemos:

`miModel.crearCoche("citroen", "SDF 1234")`: Crear un coche

`miModel.getCoche("SDF 1234")`: Nos devuelve el coche que tiene esa matrícula

Es el momento de hacer tests para saber que nuestro esquema está funcionando y es bastante flexible, además de permitirnos ampliar paulatinamente nuestra aplicación.

Vista

Ahora viene la parte de que lo que el usuario haga en la vista repercuta en el modelo. La primicia:

La Vista se relaciona con el Modelo por el Controlador

Lo primero es crear la clase View.

Añadiremos un método para mostrar la velocidad.

```
public boolean muestraVelocidad(String matricula, Integer v){  
    System.out.println(matricula + ": " + v + "km/hr");  
    return true;  
}
```

Este método lo llamaremos desde el controller cada vez que queramos mostrar la velocidad.